NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

ALEXANDRE MIGUEL DA SILVA PERES

BSc in Computer Science and Engineering

PLATFORM FOR DISSERTATION/PROJECT PROPOSAL AND MANAGEMENT

Dissertation Plan
MASTER IN COMPUTER SCIENCE AND ENGINEERING
SPECIALIZATION IN PROGRAMMING LANGUAGES AND SOFTWARE SYSTEMS

NOVA University Lisbon
Draft: *January 29, 2026*

PLATFORM FOR DISSERTATION/PROJECT PROPOSAL AND MANAGEMENT

ALEXANDRE MIGUEL DA SILVA PERES

BSc in Computer Science and Engineering

Adviser: Nuno Preguiça
Full Professor, NOVA University Lisbon

Co-adviser: Vítor Duarte
Assistant Professor, NOVA University Lisbon

Dissertation Plan
MASTER IN COMPUTER SCIENCE AND ENGINEERING
SPECIALIZATION IN PROGRAMMING LANGUAGES AND SOFTWARE SYSTEMS

NOVA University Lisbon
Draft: *January 29, 2026*

Abstract

Regardless of the language in which the dissertation is written, usually there are at least two abstracts: one abstract in the same language as the main text, and another abstract in some other language.

The abstracts' order varies with the school. If your school has specific regulations concerning the abstracts' order, the NOVAthesis LaTeX (novathesis) (LaTeX) template will respect them. Otherwise, the default rule in the novathesis template is to have in first place the abstract in *the same language as main text*, and then the abstract in *the other language*. For example, if the dissertation is written in Portuguese, the abstracts' order will be first Portuguese and then English, followed by the main text in Portuguese. If the dissertation is written in English, the abstracts' order will be first English and then Portuguese, followed by the main text in English. However, this order can be customized by adding one of the following to the file `5_packages.tex`.

```
\ntsetup{abstractorder={<LANG_1>,...,<LANG_N>}}
\ntsetup{abstractorder={<MAIN_LANG>={<LANG_1>,...,<LANG_N>}}}
```

For example, for a main document written in German with abstracts written in German, English and Italian (by this order) use:

```
\ntsetup{abstractorder={de={de,en,it}}}
```

Concerning its contents, the abstracts should not exceed one page and may answer the following questions (it is essential to adapt to the usual practices of your scientific area):

1. What is the problem?
2. Why is this problem interesting/challenging?
3. What is the proposed approach/solution/contribution?
4. What results (implications/consequences) from the solution?

Keywords: One keyword · Another keyword · Yet another keyword · One keyword more · The last keyword

Resumo

Independentemente da língua em que a dissertação está escrita, geralmente esta contém pelo menos dois resumos: um resumo na mesma língua do texto principal e outro resumo numa outra língua.

A ordem dos resumos varia de acordo com a escola. Se a sua escola tiver regulamentos específicos sobre a ordem dos resumos, o template (LaTeX) novathesis irá respeitá-los. Caso contrário, a regra padrão no template novathesis é ter em primeiro lugar o resumo *no mesmo idioma do texto principal* e depois o resumo *no outro idioma*. Por exemplo, se a dissertação for escrita em português, a ordem dos resumos será primeiro o português e depois o inglês, seguido do texto principal em português. Se a dissertação for escrita em inglês, a ordem dos resumos será primeiro em inglês e depois em português, seguida do texto principal em inglês. No entanto, esse pedido pode ser personalizado adicionando um dos seguintes ao arquivo `5_packages.tex`.

```
\abstractorder(<MAIN_LANG>):={<LANG_1>,...,<LANG_N>}
```

Por exemplo, para um documento escrito em Alemão com resumos em Alemão, Inglês e Italiano (por esta ordem), pode usar-se:

```
\ntsetup{abstractorder={de={de,en,it}}}
```

Relativamente ao seu conteúdo, os resumos não devem ultrapassar uma página e frequentemente tentam responder às seguintes questões (é imprescindível a adaptação às práticas habituais da sua área científica):

1. Qual é o problema?
2. Porque é que é um problema interessante/desafiante?
3. Qual é a proposta de abordagem/solução?
4. Quais são as consequências/resultados da solução proposta?

Palavras-chave: Primeira palavra-chave · Outra palavra-chave · Mais uma palavra-chave · A última palavra-chave

Contents

Acronyms **xi**

1 Introduction **1**
1.1 Context and Background 1
1.2 Problem Statement 1
1.3 Objectives 3
1.4 Methodology 3

2 State of the art and Technologies **5**
2.1 Backend Architectures and BaaS 5
2.1.1 Supabase (Backend-as-a-Service) 6
2.1.2 Custom Backend: Spring Boot 6
2.1.3 Custom Backend: Express.js (Node.js) 7
2.2 Data Persistence: SQL vs. NoSQL 7
2.2.1 The Relational Model (SQL) 7
2.2.2 The Non-Relational Model (NoSQL) 8
2.2.3 Justification for the Chosen Approach (PostgreSQL) 9
2.3 Frontend Frameworks and User Experience 10
2.3.1 React.js: The Foundation for Vibe Coding 10
2.3.2 Next.js: Production-Grade Framework 10
2.3.3 Tailwind CSS: Utility-First Styling 11
2.3.4 Real-Time Feedback and Interactivity 12
2.3.5 Communication Experience (Email UX) 12
2.4 AI-Assisted Development (Vibe Coding) 13
2.4.1 Concept and Capabilities 13
2.4.2 Analysis of Vibe Coding Tools 13
2.4.3 Limitations and Architectural Implications 13

3 Requirements and Current System Analysis **15**
3.1 Analysis of the Legacy System 15

3.1.1 Infrastructure and Tech Debt . . . 15
3.1.2 Functionalities and Limitations per Role . . . 16
3.1.3 Data and Reports . . . 16
3.2 Requirements Gathering . . . 17
3.2.1 Elicitation Methods . . . 17
3.2.2 Stakeholders . . . 18
3.2.3 Functional Requirements . . . 19
3.2.4 Non-Functional Requirements . . . 21
3.3 Use Cases . . . 24
3.3.1 Actors and Their Goals . . . 24
3.3.2 High-Level Use Case Diagram . . . 25
3.3.3 Detailed Use Case Specifications . . . 25
3.3.4 Use Case Relationships and Extensions . . . 25
3.3.5 Summary of Use Cases . . . 25

4 Solution Proposal **27**
4.1 Proposed System Architecture . . . 27
4.2 Process Modelling (BPMN) . . . 27
4.3 Data Model (ERD) . . . 27
4.4 API and Component Design . . . 27
4.5 Work Plan . . . 27

Bibliography **29**

Annexes

I Annex 1 - Functional and Non-Functional Requirements **35**

List of Figures

List of Tables

I.1 Functional Requirements - Edition and Configuration Management 35
I.2 Functional Requirements - Proposal Submission and Classification 36
I.3 Functional Requirements - Student Candidacies and Seriation 36
I.4 Functional Requirements - Company and User Management 37
I.5 Functional Requirements - Communication and Notifications 37
I.6 Functional Requirements - Reporting and Data Export 38
I.7 Functional Requirements - Master's Dissertation Lifecycle Support 38
I.8 Non-Functional Requirements - Security 39
I.9 Non-Functional Requirements - Auditability & Compliance 39
I.10 Non-Functional Requirements - Reliability & Availability 39
I.11 Non-Functional Requirements - Performance & Scalability 40
I.12 Non-Functional Requirements - Usability & Accessibility 40
I.13 Non-Functional Requirements - Interoperability & Data Exchange 40
I.14 Non-Functional Requirements - Maintainability & Extensibility 41
I.15 Non-Functional Requirements - Email Deliverability (Critical Legacy Fix) . . 41

Acronyms

ACID Atomicity, Consistency, Isolation, Durability *(pp. 7, 8)*
AI Artificial Intelligence *(pp. 4, 5, 10, 11)*
API application programming interface *(pp. 5–7, 9, 11)*

BaaS Backend-as-a-Service *(pp. v, 4–6)*
BASE Basically Available, Soft State, Eventually Consistent *(pp. 7, 9)*
BPMN Business Process Model and Notation *(p. 3)*

CRUD Create, Read, Update, Delete *(p. 6)*
CSS Cascading Style Sheets *(p. 11)*
CSV Comma-Separated Values *(p. 16)*

DI Department of Computer Science *(pp. 1, 3, 5, 8, 9, 15)*

EJS Embedded JavaScript *(p. 7)*
ERD Entitiy-Relationships Diagrams *(p. 3)*

FCT NOVA School of Science and Technology *(pp. 1, 3, 5, 15)*

HTML HyperText Markup Language *(pp. 7, 11, 12)*
HTTP Hypertext Transfer Protocol *(p. 15)*

I/O Input/Output *(p. 7)*
ISR Incremental Static Regeneration *(p. 11)*

JDBC Java Database Connectivity *(p. 12)*
JPA Java Persitence API *(pp. 6, 9)*
JSON JavaScript Object Notation *(pp. 8, 9, 16)*
JSONB JavaScript Object Notation Binary *(p. 8)*

JSX JavaScriptXML *(p. 10)*

LLM large language model *(pp. 5, 13)*

NoSQL Not only SQL *(pp. v, 4, 7–9)*
NOVA NOVA University Lisbon *(pp. 1, 3, 5, 15)*
novathesis NOVAthesis LaTeX *(pp. i, iii)*

RBAC role-based access control *(p. 22)*
RLS Row-Level-Security *(pp. 6, 9, 22)*

SPA Single Page Application *(p. 10)*
SQL Structured Query Language *(pp. v, 4, 6–9)*
SSG Static Site Generation *(p. 11)*
SSL Secure Sockets Layer *(p. 15)*
SSR Server-Side-Rendering *(p. 11)*
STOMP Simple Text Oriented Messaging Protocol *(p. 12)*

TLS Transport Layer Security *(p. 15)*

UI User Interface *(pp. 10–13, 16)*
UX User Experience *(pp. 2, 4, 10)*

WYSIWYG What You See Is What You Get *(pp. 12, 16)*

1 Introduction

1.1 Context and Background

The transition of students from academic life to the professional market or advanced research is a centerpiece of the university experience, essentially enabled through three different distinct paths at the Department of Computer Science (DI) of NOVA School of Science and Technology (FCT):

- Undergraduate Internships: Primarily offered by external companies to provide students with their first professional experience.
- Master's Dissertations: Scientific thesis typically conducted within the university, focusing on academic research under the supervision of professors.
- Engineering Projects: Practical projects proposed by external companies or the Department of Informatics itself, allowing Master's students to apply advanced engineering principles to real-world projects.

Managing those processes involves a complex coordination of multiple stakeholders: students seeking opportunities aligned with their skills and interests, professors overseeing academic quality, and external companies providing practical challenges. Within this ecosystem, a reliable digital platform is essential to act as a central hub for communication, proposal submission, and student placement.

1.2 Problem Statement

At DI @NOVA FCT, the management of internships, dissertations and engineering projects involves a multi-stage lifecycle that includes proposal submission, clarification and approval processes, making proposals available to students, report/dissertation submission, and providing documentation to juries. Currently, the department relies on a legacy platform to support some of these activities, however this system presents several critical issues that hinder the efficiency of the department.

The most significant limitation is that the current platform is not being utilized for the Master's degree processes - including both scientific dissertations and engineering projects. At the moment it is exclusively used to manage undergraduate internships, meaning that the more complex workflows associated with Master's students are handled through manual, or non-standardized methods. This creates a lack of centralized data an increases the administrative burden on coordinators, professors and the secretariat.
Furthermore, even within its current scope, the legacy system suffers from several functional and technological gaps:

- Communication Gaps and Reliability: The current method for "Call for Proposals" is manual, requiring coordinators to send individual emails to companies, advertising the opportunity send proposals, for each edition. There are no safeguards to ensure that these emails follow specific deliverability rules, often resulting in messages being flagged as spam. Additionally, the system lacks the flexibility to create and edit email templates, preventing coordinators from customizing communication for different editions or context.

- Inefficient Feedback Loops: Although companies can submit proposals, there is no integrated mechanism for coordinators to provide direct feedback on those proposals for clarification. This forces corrections to be handled outside the platform, whereas a centralized system would allow companies to correct and resubmit proposals based on coordinator comments.

- Stale Company Data: The existing repository contains a large amount of outdated information, including companies that no longer exist or have changed their contacts or other information. There is a lack of a "validation" status to ensure only active and verified companies can send proposals.

- Manual Protocol Management: For new companies that have never participated in previous editions, the process of signing the required protocols is still entirely manual. The platform should automate delivery and tracking of these legal documents as soon as a new company registers.

- Outdated User Experience (UX): The interface is not intuitive for external stakeholders, making it difficult for companies to register, manage their employees accounts, or monitor the status of their proposals in real-time.

In conclusion, the necessity for a new platform arises from the need to unify all academic degrees under a single digital infrastructure, replacing manual tracking with a secure, automated system that ensures data integrity and seamless experience for all stakeholders.

1.3 Objectives

The primary objective of this dissertation is to design and develop a modernized web-based platform that unifies and automates the management and entire lifecycle of undergraduate internships, Master's dissertations, and engineering projects for DI@NOVA FCT. The projects aim to transition from a fragmented, manual system to a centralized digital ecosystem.
To achieve this goal, the following specific objectives have been defined for the initial version:

- Process Unification: Integrate the lifecycles of undergraduate internships and Master's degree pathways (both scientific and engineering-focused) into a single platform, replacing the current manual methods.
- Workflow Automation: Implement automated procedures for the "Call for Proposals", student seriation (ranking), and the validation of proposals.
- Enhanced Communication and Feedback: Establish direct feedback loops within the platform, allowing coordinators to request clarifications on proposals and ensuring companies can resubmit corrected versions.
- Document and Protocol Management: Automate the delivery, tracking, and storage of legal protocols, student CVs, and recommendation letters.
- Administrative Oversight: Develop advanced administrative tools, including "superuser" access for student support, detailed system logs for auditing, and validation workflow for new company registrations.

1.4 Methodology

This project will follow an Agile/Scrum methodology, characterized by iterative development, frequent feedback loops [40] to ensure the final platform aligns with the university's complex requirements. The work plan is structured into several phases, prioritizing rapid prototyping followed by a transition to a high-performance custom architecture.
To achieve this goal, the following specific objectives have been defined for the initial version:

1. Requirement Gathering and Analysis: A thorough analysis of the legacy system and current manual processes was conducted to identify functional and non-functional requirements. This phase includes the definition of user roles (Admin, Coordinator, Student, Professor, Company, Secretariat) and their respective use cases.
2. System Modeling and Design: Utilizing Business Process Model and Notation (BPMN) to map complex workflows [28] and Entitiy-Relationships Diagrams (ERD)

to design the database schema [7]. This phase also involves planning the system architecture.

3. AI-Assisted Prototyping (Vibe Coding): The initial implementation phase will leverage Vibe Coding tools, such as Lovable, Figma Make or Google AI Studio, to rapidly generate the frontend and core functional interfaces [9]. These tools excel at creating React-based user interfaces through iterative natural language prompting. During this stage, Supabase will be utilized as Backend-as-a-Service (BaaS) to provide an immediate infrastructure for authentication, database management, and initial data structures via SQL[50]. This approach allows for the rapid fulfillment of most functional requirements while maintaining a live, testable version of the platform.

4. Architectural Transition and Manual Development: Once the prototype reaches a stable state and meets the primary functional requirements, the project will transition from AI-generated code to manual development. This move is essential to refine the User Experience, optimize performance, and implement custom features that go beyond the limitations of AI-driven boilerplate.

5. Custom Backend Implementation: To support the department's more complex business logic - such as the specialized student seriation (ranking) algorithms and robust administrative auditing - the system will transition to a dedicated custom backend, likely using Spring Boot [43]. While Supabase is effective for rapid prototyping, a Spring Boot architecture offers superior domain modeling, type safety and maintainability through its layered structure (Controller, Service, Repository). This transition ensures that the platform is not only fast to develop initially but also scalable and reliable for long-term university use.

6. Verification and Validation: The platform will be tested against the initial requirements, ensuring that automated emails follow specific deliverability rules avoiding being flagged as spam. Additionally, the seriation logic will be rigorously validated to ensure it correctly handles student placements and matching according to pre-defined criteria.

7. Documentation and Evaluation: There will be a continuous documentation of the architecture decisions, providing detailed justifications for technical choices such as the preference for SQL over NoSQL for structured academic data, and other technical choices.

2

State of the art and Technologies

This chapter examines the current state of software engineering practices and technologies relevant to building a modern platform for the DI at NOVA FCT. The goal is to overcome limitations of the legacy system - such as fragmented data, manual workflows, and inefficient communication - by systematically evaluating contemporary approaches to backend architectures, data persistence, frontend frameworks, and AI-assisted development workflows. Particular attention is given to the emerging paradigm of AI-assisted software development, often described as "Vibe Coding" [14] which supports rapid prototyping by allowing developers to steer large language models (LLMs) through natural-language interactions instead of traditional line-by-line coding.

Modern web application stacks are characterized by a wide range of choices at each layer, from infrastructure and backend services to frontend frameworks and development workflows. In the backend, developers must choose between fully managed BaaS platforms [17] and custom backends that offer fine-grained control over domain logic, performance, and governance. On the frontend, single-page applications and component-based frameworks (such as React, Angular, or Vue) are commonly combined with RESTful or GraphQL APIs, enabling decoupled client-server architectures and iterative delivery of new features.

Beyond classical architectures, AI-assisted development tools and practices are reshaping how software is designed and implemented. Under the "Vibe Coding" paradigm, developers describe desired functionalities to an LLM, which generates and refines code iteratively, prioritizing rapid experimentation and high-level intent over manual code authoring. This approach align with agile or small teams to assemble complex prototypes quickly, though it raises questions about maintainability, security, and auditability in production systems. [39]

2.1 Backend Architectures and BaaS

The choice of backend architecture directly influences development speed, scalability, operational complexity, and the ability to encode complex business rules such as student

seriation and multirole access control. For this project, two main paradigms are considered: Backend-as-a-Service solutions, which offer ready-made infrastructure and managed services, and custom backends built with mature frameworks such as Spring Boot and Express.js, which provide full control over domain modeling, integration patterns, and deployment environments. [5]

BaaS platforms typically optimize for rapid time-to-market by abstracting away server provisioning, database management, and authentication, making them well-suited for early-stage prototyping and small teams [1]. In contrast, custom backends allow tailoring of data models, ownerships of the entire codebase, and fine-grained observability, which becomes critical for institutional applications that must satisfy compliance, long-term maintainability, and integration with existing academic processes.

2.1.1 Supabase (Backend-as-a-Service)

Supabase is an open-source BaaS stack that builds on PostgreSQL and exposes automatically generated REST APIs through PostgREST [48], significantly reducing boilerplate CRUD implementation effort. By providing an integrated suite of tools - including database, authentication, storage, and real-time capabilities - Supabase enables rapid development cycles while leveraging a relational data model that is compatible with established SQL tooling and practices. [50]

A central feature of Supabase is its built-in authentication system, which supports user registration, password-based login, magic links, and role-based access control[44], simplifying the enforcement of different permission profiles for students, companies, professors and administrators. Row-Level-Security (RLS) policies can be defined directly at the database level, ensuring that authorization rules are enforced close to the data and cannot be bypassed by misconfigured API clients [49]. For logic that goes beyond standard CRUD operations - such as orchestrating notification workflows or managing student seriation - Supabase offers serverless "Edge Functions", typically written in TypeScript or JavaScript, which run in a managed environment near end users to minimize latency. [46]

2.1.2 Custom Backend: Spring Boot

Spring Boot is a widely adopted framework for building production-grade Java or Kotlin backends and is especially suitable for complex, domain-driven systems that require strict typing, layered architectures, and extensive integration capabilities. Its opinionated configuration model and extensive ecosystem (Spring Data, Spring Security, Spring Cloud) allows developers to structure applications into controllers, services, and repositories, facilitating clear separation of concerns and maintainable domain models. [43]

For academic workflows that must support student ranking algorithms, complex eligibility rules, and auditable decision-making processes, the type safety and testability provided by Spring Boot are particularly advantageous. The framework integrates with JPA and tools like Hibernate Envers to support automatic auditing of entity changes [43,

6], which helps track user actions, state transitions, and login-related events - capabilities that are often required for administrative oversight and internal compliance in university environments.

2.1.3 Custom Backend: Express.js (Node.js)

Express.js is a minimalist web framework for Node.js that offers a lightweight and flexible foundation for building HTTP APIs using JavaScript or TypeScript [13]. Its event-driven, non-blocking I/O model allows a single process to handle many concurrent connections efficiently, making it well-suited for notification-heavy workloads [27] such as broadcasting "Call for Proposals" or status updates to large groups of students.

The Node.js ecosystem provides a rich set of libraries for templated email generation and delivery, including tools such as Handlebars or EJS to render dynamic HTML email bodies [18, 12] for coordinators and administrators. Combined with tasks schedulers or message queues, Express-based backends can orchestrate asynchronous communications and background jobs [27], helping address a key functional gap of legacy systems that rely on manual email workflows and ad hoc templates.

2.2 Data Persistence: SQL vs. NoSQL

The selection of a data persistence model is a fundamental architectural decision that dictates how the system ensures data integrity, scalability, and retrieval efficiency [54, 4]. Given the complex ecosystem of the thesis and internships management, which involves multiple stakeholders (Students, Professors, Companies, Secretariat, etc...) and multi-stage lifecycle workflows for academic projects, this section evaluates the trade-offs between Relational (SQL) and Non-Relational (NoSQL) databases.

The choice between SQL and NoSQL has complex implications for institutional systems. Relational databases prioritize data integrity and consistency through Atomicity, Consistency, Isolation, Durability (ACID) compliance [35], while Non-Relational systems typically follow the Basically Available, Soft State, Eventually Consistent (BASE) model, trading immediate consistency for horizontal scalability and schema flexibility [53]. For academic management platforms where data accuracy are indispensable, this distinction is critical.

2.2.1 The Relational Model (SQL)

Relational databases structure data into tables with predefined schemas, utilizing Foreign Keys to establish strict relationships between entities [19]. This approach provides fundamental guarantees about data quality and consistency that are essential for institutional systems.

- Structured Data and Integrity: The academic context requires rigorous data consistency. For instance, a Candidacy must strictly link to an existing Student and a valid Proposal. The relational model enforces referential integrity through primary and foreign key constraints, preventing "orphaned" records - a situation where a child record references a non-existent parent record [41]. This is particularly critical in the DI management platform, where deleted Companies should not retain active Proposals, and Students cannot be assigned to non-existent projects.

- Referential integrity mechanism ensure that every foreign key in a child table corresponds to a valid primary key in the parent table, maintaining logical consistency across the entire database. Beyond preventing data anomalies, referential integrity supports cascading updates and deletions, ensuring that changes propagate correctly across related entities [19, 41].

- Complex Querying: The platform requires complex join operations to generate views - for example, listing all proposals for a specific edition filtered by a company's validation status, or retrieving all students that are candidates to a particular internship. SQL excels at these complex relationships through its mature and optimized query language, which can efficiently traverse multiple table relationships in a single query [24].

- PostgreSQL: This project will initially utilize PostgreSQL (via Supabase for rapid prototyping). Beyond standard SQL features, PostgreSQL offers support for complex data types (JSON/JSONB, arrays, enums) and Stored Procedures [29, 33]. This allows the system to handle semi-structured data - such as like flexible feedback forms or email template configurations - while maintaining the benefits of a relational schema for academic entities.

2.2.2 The Non-Relational Model (NoSQL)

NoSQL databases (e.g., document stores like MongoDB or key-value systems like Redis) offer schema flexibility and horizontal scalability, often at the cost of immediate consistency (ACID properties) [54]

- Flexibility vs Structure: While NoSQL allows for rapid iteration of data models without schema migrations, the academic domain is highly structured. Entities such as Users, Editions, Proposals, Companies and Students have well-defined attributes that rarely change dynamically or unexpectedly. The governance requirements of an academic institution demand clear, predictable data structures rather than evolving, loosely-typed records.

- Consistency Challenges: In workflows like Student Seriation, data consistency is essential. A NoSQL approach, which often relies on eventual consistency, could introduce race conditions where a student is assigned to a project that is no longer

available or where multiple simultaneous requests lead to conflicting state changes. The BASE model prioritizes availability and partition tolerance over strict consistency, which is acceptable for read-heavy analytics systems but unsuitable for transactional workloads that require immediate correctness.

2.2.3 Justification for the Chosen Approach (PostgreSQL)

The decision to utilize PostgreSQL is driven by the specific requirements and constraints of the DI management platform:

1. Data Integrity and Referential Consistency: The need to enforce relationships between Companies, Proposals, Editions, and Students needs a relational schema with foreign key constraints. For example, the system must prevent an inactive or deleted company from having active proposals, and it must prevent logically impossible states such as student being assigned to multiple conflicting projects in the same edition. These constraints are best expressed and enforced at the database level through relational integrity mechanisms rather than dispersed throughout the platform's code [41, 19].

2. Row-Level-Security (RLS) and Multi-Tenancy: Using PostgreSQL enables the implementation of RLS, a database-level access control mechanism. RLS policies allow fine-grained permissions to be defined directly in the database layer, ensuring that access rules (e.g. "a company can only see the candidates to its own proposals", "a student can only see the applications they've made") are enforced uniformly regardless of how the API is accessed. This provides a security layer that cannot be bypassed by the application logic, addressing a critical requirement for institutional data protection, By using PostgreSQL's RLS policies with role-based contexts, the platform achieves secure multi-tenancy at the database level, without relying solely on application-level permission checks. [16]

3. Hybrid Capabilities and Schema Evolution: PostgreSQL provides the strict structure required for academic records while support JSON types, effectively making the for using a NoSQL solution unnecessary. The hybrid approach gives the platform the best of both worlds: schema enforcement where it matters (core academic entities and relationships) and flexibility where needed (dynamic form configurations, email templates, audit logs). This design reduces operational complexity by consolidating storage into a single database system while maintaining both consistency guarantees and the agility to evolve data models as requirements change. [29]

4. Compatibility and Technology Continuity: Both the rapid prototyping tools (Supabase [45]) and the target production framework (Spring Boot with JPA/Hibernate [42]) are optimized for relational databases, ensuring a smooth transition between development phases. This alignment minimizes architectural friction and ensures

that the knowledge base and tooling remains consistent as the system evolves from prototype to production.

2.3 Frontend Frameworks and User Experience

The "Outdated User Experience (UX)" of the legacy system is identified as a critical failure point, particularly for external stakeholders such as companies who struggle to register, manage accounts, or monitor proposal status. To address these limitations, the new platform must transition from static, server-rendered pages to a modern, reacting Single Page Application (SPA) or hybrid architecture that enables faster navigation, richer interactivity [36], and consistent interfaces across distinct user roles.

2.3.1 React.js: The Foundation for Vibe Coding

React is a free, open-source JavaScript library maintained by Meta that specializes in building interactive user interfaces through component-based architecture [36, 37]. Unlike traditional web development where the entire page must be reloaded on data changes, React efficiently re-renders only the components that have actually changed, using its virtual DOM reconciliation process to minimize performance overhead [25]. React uses JavaScriptXML (JSX), a syntax extension that allows developers to write HTML-like code directly within JavaScript [36], making the UI logic more readable and maintainable.

React is selected as the core library for building the platform's user interface for two primary reasons:

1. Component-Based Architecture: React allows for the creation of reusable, self-contained UI components (e.g. "ProposalCards" where all the details from a proposal are, that can be used in multiple pages) [34]. This modular approach ensures consistency across the distinct interfaces required for Students, Professors, and Companies, while enabling efficient code reuse and simplified maintenance.

2. Synergy with AI-Assisted Development: The project methodology relies on Vibe Coding tools for rapid prototyping. Tools such as Lovable, Bolt.new, Figma Make and Google AI Studio are optimized to generate code specifically for the React ecosystem. Using React ensures that the code generated during the prototyping phase is immediately usable and extensible, eliminating the need for time-consuming library migrations.

2.3.2 Next.js: Production-Grade Framework

Next.js is a full-stack React framework created and maintained by Vercel that extends React's capabilities [26] by providing built-in solutions for routing, data fetching, server-side rendering, and infrastructure - all without complex configuration. While React is a library focused solely on the UI layer and requires additional libraries for routing and

server-side concerns, Next.js provides an integrated toolkit that handles both frontend rendering and backend API logic in a single project [26]. Next.js supports multiple rendering strategies: Server-Side-Rendering (SSR) for dynamic content rendered on-demand, Static Site Generation (SSG) for pre-rendering at build time, and Incremental Static Regeneration (ISR) for updating static content without full rebuilds [23].

Next.js is selected to complement React because it provides essential production-grade features:

- Routing and Performance: Next.js offers a file-system based router (App Router) [3] and rendering strategies such as SSR and SSG. SSR renders pages on demand for dynamic content, while SSG pre-renders static pages at build time, reducing server load times and enabling fast initial page loads [23], which is crucial for retaining the engagement of external company users. This flexibility allows different sections of the platform to use the optimal rendering strategy based on content volatility.

- Scalability and Full-Stack Capabilities: As the platform transitions from a prototype to a long-term solution, Next.js provides the architectural structure needed to manage complex API integrations, and full-stack workflows. In initial phases, Next.js API routes can serve as lightweight backend layer [2] before the full transition to Spring Boot, reducing deployment complexity during the transition period.

2.3.3 Tailwind CSS: Utility-First Styling

Tailwind CSS is a utility-first CSS framework that provides a comprehensive set of low-level, single-purpose utility classes (such as `p-4` for padding, `text-center` for text alignment, or `bg-blue-600` for background color) that can be composed directly in HTML markup without writing custom CSS. Unlike traditional CSS frameworks (such as Bootstrap) that provide pre-built UI components, Tailwind CSS empowers developers to build custom designs by combining small utility classes, giving them complete design control while maintaining consistency [38].

Tailwind CSS is selected for styling the platform for two key reasons:

- Rapid UI Development: Tailwind utility-first approach enables developers to style elements directly in HTML markup without writing custom CSS. This significantly accelerates the conversion of "mental models" or Figma designs into working code, reducing the time spent switching between markup/code and style files [38].

- AI Integration: Most AI coding assistants (specifically Lovable) default to generating Tailwind CSS code. This ensures that the aesthetic output from the Vibe Coding process is modern, responsive, and easy to maintain without the complexity of managing large external CSS files.

2.3.4 Real-Time Feedback and Interactivity

The legacy system suffers from "Inefficient Feedback Loops" where users must constantly refresh or wait for emails to know the status of a proposal.

- Supabase Realtime (Prototyping Phase): During rapid prototyping with Supabase, Real-time subscriptions powered by WebSockets, allow the frontend to listen to database changes instantly and update the UI components without requiring page refreshes [47]. For example, a student receives an immediate "Selected" when a company confirms an interview result, this improves user engagement.

- Production Architecture: PostgreSQL LISTEN/NOTIFY with Spring Boot WebSocket: When transitioning to Spring Boot, real-time updates leverage PostgreSQL's native LISTEN/NOTIFY mechanism - a lightweight, built-in pub/sub system [32, 31] that requires no external message brokers. The workflow is straightforward.

  1. Database Trigger: When a Proposal status changes (e.g., from "Pending" to "Accepted") a PostgreSQL trigger automatically fires and sends a notification via `pg_notify()` to a named channel [32, 31].
  2. Spring Boot Listener: A background thread in Spring Boot continuously listens on the PostgreSQL notification channel using JDBC's PostgreSQL driver extensions [20]: When a notification is received, it triggers an event in the application.
  3. WebSocket Broadcast: Spring Boot WebSocket infrastructure (optionally using Simple Text Oriented Messaging Protocol (STOMP) for advanced message routing) [52, 43] broadcasts the update to all connected frontend clients subscribed to relevant topics. For instance, all students viewing proposals from a specific company receive live updates when that company changes a proposal's status.

2.3.5 Communication Experience (Email UX)

A major pain point is the lack of flexibility in creating email templates and the issue of emails being flagged as spam.

- Template Management: The frontend will feature a What You See Is What You Get (WYSIWYG) markdown or HTML editor [30] (integrated into the Admin Dashboard) allowing coordinators to visually edit email templates, without the knowledge of those markup languages. This eliminates the need for manual template coding and reduces errors.

- Deliverability: To ensure emails reach users inboxes, the system architecture must integrate with an established transaction email service rather than relying on the error-prone, manual email workflows of the legacy system [22]. These services

provide authentication protocols, monitoring, and detailed analytics to maintain sender reputation and maximize inbox placement [10].

2.4 AI-Assisted Development (Vibe Coding)

Traditional software development involves writing code manually, line by line. However, a new paradigm known as "Vibe Coding" has emerged, leveraging large language models (LLMs) to generate full-stack applications or complex User Interface (UI) components through natural language prompting [8, 39]. This section analyzes the capabilities of these tools to determine their viability for the rapid prototyping phase of the dissertation.

2.4.1 Concept and Capabilities

Vibe Coding tools differ from standard code assistants (like GitHub Copilot) by focusing on generating entire applications or functional modules rather than just autocomplete syntax. They typically integrate a development environment, a live preview window, and an AI agent capable of iterating on code based on user feedback. The primary advantage of this approach is the drastic reduction in time required to move from a conceptual requirement to a testable, interactive interface.

2.4.2 Analysis of Vibe Coding Tools

TODO - need to check the tools better because since I was researching they got updated and some new tools appeared too

2.4.3 Limitations and Architectural Implications

TODO - waiting for the subsection before

3

Requirements and Current System Analysis

3.1 Analysis of the Legacy System

The current internship management platform at DI @ NOVA FCT is a web-based solution built on a deprecated technology stack. Although it supports the undergraduate internship life cycle, it presents critical limitations regarding security, architecture, and functionality. The section provides a systematic analysis of the current system's components and deficiencies, informed by direct exploration of the platform.

3.1.1 Infrastructure and Tech Debt

The system is hosted on a virtual machine and operates on an obsolete version of PHP dating back to approximately 2010, but since the platform was made even before that maybe there are some modules that are way older. This legacy codebase entails high maintenance risks and incompatibilities with modern libraries. Specifically, older PHP versions no longer receive security updates, leaving the platform vulnerable to known exploits that malicious actors routinely target [11].

A more critical vulnerability lies in the platform's network architecture: the system communicates exclusively over HTTP (without SSL/TLS encryption), representing a severe risk for transmitting sensitive student data and access credentials. Data transmitted over unencrypted HTTP is exposed to interception and eavesdropping, allowing attackers to read login credentials, personal information, and institutional data in plaintext [21].

Email Deliverability Crisis: A particularly severe technical deficiency resides in the email infrastructure. Due to the pack of modern authentication and deliverability configurations, transactional emails sent by the platform - intended for registration notifications, deadline reminders, call for proposals - are systematically flagged as spam by email providers [10]. This breakdown in communication disrupts the workflow between the department, students, and companies, reducing the operational effectiveness of the platform regardless of other functionality.

3.1.2 Functionalities and Limitations per Role

Administration and Coordination: The administrator profile provides a global overview of entities and proposals, allowing actions such as bulk student imports via CSV and basic permission management. A “superuser” feature exists, allowing administrators to access any user account without credentials for troubleshooting purposes. However, a critical gap exists in audit logging: while the system logs normal and superuser’s logins and logouts, it lacks comprehensive action login that tracks what users do during their sessions. For compliance and accountability, the platform should record specific administrative actions such as proposal status changes, student data modifications, permissions adjustments, and bulk import operations.

Content management relies on outdated, inflexible in-browser WYSIWYG editors for maintaining FAQs, Regulations, and Edition-specific information. Mass communication is rigid: the system offers no configurable templates for different stages of the workflow, relying instead on a single standard message for inviting companies.

Proposal Management and the Master thesis Gap: The platform includes interfaces for companies or professors to submit internship proposals and Master’s dissertation themes. However, these dissertation theme submission and management workflows are completely inoperative - they exist as “dead data” in the system, forcing the department to manage dissertation assignments, publication and monitoring via external tools, negating the value of having collected this data in the platform.

For undergraduate internships, the “Project Classification” workflow allows coordinators to accept, reject, or request clarifications by annotating proposals. However, interaction remains limited: coordinators can toggle whether a proposal awaits company response, but the feedback mechanism lacks fluid integration for iterative proposal refinement.

Students and Companies: The student interface allows ranking of internship preferences and the consultation of imported academic records. For companies, human resources management is inefficient: there is no clear hierarchy or efficient mechanisms for managing multiple employees within a single company entity. The overall UI is outdated, hindering navigation and making it difficult for users to perceive current application status.

3.1.3 Data and Reports

The current system suffers from a critical lack of interoperability. There are no native features to export complex data (student lists, proposals, results of attributions) into open, standard format such as CSV or JSON, which complicates operations that the coordinators might need to make with the data outside the platform. Furthermore, company information is often outdated, as there are no periodic revalidation mechanisms to refresh company profiles and contact details ("Stale Company Data").

3.2 Requirements Gathering

Following the systematic analysis of the legacy platform and the identification of its critical failures, a comprehensive set of requirements was elicited for the new integrated platform. The elicitation process focused on addressing the needs of the diverse stakeholders involved in the academic lifecycle (students, companies, coordinators, secretariat, and administrators), with the goal of transitioning from fragmented spreadsheet-based management to a unified digital ecosystem supporting both undergraduate internships and master's dissertations. The resulting requirements were prioritized and refined in collaboration with the thesis advisers to ensure alignment with institutional constraints regarding data protection, interoperability, and operation feasibility.

3.2.1 Elicitation Methods

To ensure the new platform addresses real operational pain points and does not introduce new barriers to existing workflows, requirements were gathered through a multi-faceted approach:

- Legacy System Analysis: Direct exploration and reverse-engineering of the existing platform to identify functional gaps (e.g. non-functional Master's modules) and technical debt (e.g. email deliverability issues). This analysis, presented in Section 3.1., served as the foundation for identifying core functional requirements.

- Document Analysis: Review of current regulations, current internship rules, and manual processes currently used to manage Master's dissertations and feedback tracking. This provided insight into established business logic that must be preserved and systematized in the new platform.

- Workflow Modeling: Mapping of the "as-is" manual processes and system workflows against desired "to-be" automated and digitized workflow, Specifically for proposal submission and validation, student seriation algorithms, registrations workflows, company hierarchies, and feedback management. Process models were documented to clarify state transitions, decision points, and inter-stakeholder communication patterns.

- Stakeholder Consultations: Informal discussions with thesis advisers and the current coordinator responsible for the platform administration provided critical insights into administrative pain points, compliance requirements, and feature priorities. These discussions informed the prioritization of features and validated the proposed solution direction.

3.2.2 Stakeholders

The system must support multiple distinct user groups. However, the role hierarchy reflects the institutional structure where coordinators are designed professors with expanded administrative responsibilities, rather than a separate administrative role.

- Students: Seek to consult and review available internship and dissertation/project proposals within their enrolled edition, rank proposals according to their preferences, track the status of their candidacies and assignments, access their assigned supervisor's contact information, and, if required by the coordinators, submit documentation (CV, motivation letter, or recommendations) for specific proposals.
- Company Representatives: Aim to register company information and manage organizational hierarchy (parent companies and sub-entities or departments), submit internship and Master's dissertations proposals during open call periods, receive and respond to feedback from coordinators regarding proposal clarifications, manage company employees and designate contacts responsible for specific proposals, schedule and present their proposals during coordinator provided time slots, and track interview progress and student candidacies through a dashboard.
- Professors: Intent to proposed scientific dissertation themes for the Master's degree, supervise students assigned to their dissertation topics, provide feedback on dissertation progress.
- Edition Coordinators (Professor variant, Head Coordinator): Is a professor with comprehensive administrative responsibilities for the system. Their responsibilities include configuring and managing editions (setting edition type - undergraduate internship or Master's dissertations, defining proposals' deadlines, configuring available time slots for company presentations, creating edition-specific information pages such as FAQs and contact pages), validate submitted proposals through a feedback-and-clarification workflow, control the attributions process (being able to manually add a student to an internship outside seriation), generating and customizing email templates for all communications stages, publish news and announcements targeting different user groups, managing user pre-registration and bulk imports via CSV, and overseeing the finalization of internship/dissertation protocols. The head coordinator also maintains comprehensive audit logs and manages permissions assignments for sub-coordinators and other edition-specific administrative tasks.
- Sub-Coordinators (Professor variant): Professors designed by the head coordinator to assist with proposals validation and feedback workflows for specific edition, but without access to system-wide configuration. They inherit permissions from the Professor's role but with additional privileges limited to proposal evaluation and validation, configuration of edition-specific resources (FAQs, contact pages, email templates) within their assigned edition.

- Secretariat/Administrative Staff: Manage financial and legal formalization of internships and dissertations, specifically verifying payments from companies for protocol signing and managing storage and archival of signed agreements. They require a view-only access to status dashboards showing active internships and their completion status.

- Administrators: Handle the infrastructure-level operations that fall outside the coordinator's scope, such as system backups, database maintenance, server updates, monitoring system health, managing SSL certificates and security configurations, responding to critical technical incidents, and consulting detailed audit logs for technical troubleshooting and compliance verification. They do not manage user accounts, permissions, or edition-specific configurations - those remain the responsibility of the head coordinator. This role is typically filled by dedicated IT staff rather than a professor.

3.2.3 Functional Requirements

The functional requirements are categorized by feature set to ensure modular development, testing and clear traceability of stakeholder needs. Detailed specification for all functional requirements are presented in Annex 1, Tables I.1-I.7. The following subsections provide a summary of each category:

3.2.3.1 FR-1x: Edition and Configuration Management

This category contains the administrative infrastructure required for coordinators to establish and manage academic editions. Edition coordinators must be able to create new editions, specifying whether the edition targets undergraduate internships or Master's dissertations, and define critical timelines (proposal submission deadlines, interview periods and finalization deadlines). The system must provide tools for scheduling company presentation time slots, designing reusable email templates with dynamic variable substitution (student names, company names, deadlines), creating and maintaining edition-specific informational content (FAQs, regulations, contact information), and publishing targeted announcements to specific user roles. These capabilities replace the legacy system's outdated, inflexible approach to edition configuration and enable coordinators to adapt communication and workflows to each edition's unique needs.

3.2.3.2 FR-2x: Proposal Submission and Classification

This category defines the workflows for companies and professors to propose internship opportunities and dissertation themes. The system support detailed proposal submission with attributes such as objectives, technical requirements, location, and benefits. A critical feature is the feedback loop: coordinators can request clarifications on proposals without rejecting them outright, triggering automatic notifications to the author for editing and

resubmission, enabling a collaborative refinement process. Coordinators also retain the ability to accept, conditionally accept, or reject proposals with detailed comments. The system shall allow companies to reuse and adapt proposals from previous editions, streamlining the proposal process for returning partners, and to schedule and manage oral presentations of their proposals using the coordinator-defined time slots. Deadline notifications ensure companies remain informed of submission closures and feedback response windows.

3.2.3.3 FR-3x: Student Candidacies and Seriation

This category automates the student ranking and assignment process, addressing a core business requirement of the platform. Students must be able to view available proposals for their enrolled edition and rank a specified number of preferences in priority order. If required by coordinators, students can submit supporting documentation (CV, motivation letter, recommendation letters) to strengthen their candidacy for specific proposals. The system implements an automated seriation algorithm that ranks candidates for each proposal based on pre-defined criteria (ECTS completed, grade average, coordinator-weighted factors), producing a ranked list of candidates for company review. The system notifies students of key timeline events (registration opening, ranking deadline, seriation results, confirmation) via email and in-app notifications. Companies then view the serialized candidate lists and manage candidate selection through interview and final confirmation phases, with the ability to exclude candidates if needed.

3.2.3.4 FR-4x: Company and User Management

This category establishes user and organizational management capabilities, supporting diverse the diverse stakeholder ecosystem. Edition coordinators validate new company registrations and ensure legal protocol compliance before companies submit proposals. Company representatives manage their organizational hierarchy (designating parent companies, sub-entities, departments), maintain current contact information, and administer employee accounts (adding, removing, updating mentors and proposal-responsible personnel). The system provides superuser functionality, allowing coordinators temporary access to other user accounts using only a user identifier for support purposes, with full audit logging. Coordinators can pre-register users in bulk (students via CSV import or manual entry, professors, secretariat staff) with role-based permissions. Additionally, coordinators can assign and revoke sub-coordinator roles to professors for specific editions, enabling delegation of proposal validation and feedback responsibilities.

3.2.3.5 FR-5x: Communication and Notifications

This category addresses the legacy system's critical email communication failures by implementing reliable, templated, multichannel notification workflows. The system automates the "Call for Proposals" process, notifying all registered companies at the start

of each edition with submission deadlines and edition type. Coordinators can publish news and announcements targeted at specific user roles (students-only news, company-only alters, etc.). The system enforces email deliverability best practices (discussed in Section 3.2.4.8) to ensure notifications reach intended recipients reliably, replacing the legacy system's spam-flagging problem. Both email and in-app notifications are supported for critical events (new proposals, feedback requests, deadline reminders, seriation results). Coordinators can send targeted email campaigns to specific user groups with customizable subject and body content, enabling flexible mass communication without the rigidity of the legacy system.

3.2.3.6 FR-6x: Reporting and Data Export

This category enables coordinators and administrators to extract and analyze system data for institutional reporting and decision-making. The system allows comprehensive export of student lists, proposals, candidacies, and final assignments in open standard formats (CSV, JSON), addressing the legacy system's lack of interoperability. Detailed audit logs record all critical system actions (logins/logouts, proposal status changes, superuser access, permission modifications, bulk imports) with timestamps and user attribution, enabling accountability and compliance verification. Auditable records of proposal feedback exchanges (coordinator feedback, company responses, timestamps) are maintained for dispute resolution and quality assurance.

3.2.3.7 FR-7x: Master's Dissertation Lifecycle Support

This category operationalizes Master's degree workflows, addressing the legacy system's "dead data" problem where dissertation themes were submitted but never managed within the platform. The system distinguishes between scientific dissertations (academically-focused) and engineering projects (industry-focused), supporting tailored workflows and evaluation criteria for each type. Professors can propose scientific dissertation themes, managing their proposed themes and assigned students. The system enables progress tracking and completion status management, including key milestones (topic approval, final submission/defense). Master's editions operate similarly to undergraduate internship editions but with workflow and attribute modifications tailored to the distinct requirements of Master's level supervision and evaluation.

3.2.4 Non-Functional Requirements

To address the technical debt of the legacy system and ensure the platform meets institutional and user expectations, the new system must adhere to the following quality attributes, organized according to ISO/IEC 25010 standards. Detailed specifications for all-non functional requirements are presented in Annex 1, Tables I.8-I.15. The following subsections provide a summary of each category:

3.2.4.1 Security

Security controls ensure the platform protects sensitive institutional and personal data from unauthorized access and breaches. The system implements RLS at the database level to enforce strict data isolation, ensuring users can only access data appropriate to their role and organization context (e.g., companies see only their own proposals and candidates). All communications must be encrypted using HTTPS/TLS to prevent interception of credentials and sensitive information during transmission, directly addressing the legacy system's unencrypted HTTP vulnerability [21]. Passwords and sensitive credentials must be hashed using industry-standard algorithms and never stored in plaintext. role-based access control (RBAC) with granular permission assignment ensures users can only perform actions consistent with their assigned role [51].

3.2.4.2 Auditability & Compliance

Comprehensive audit logging enables institutional accountability, compliance verification, and dispute resolution. The system records an immutable, tamper-evident audit log of all critical actions (logins/logouts, data modifications, status changes, permission assignments, superuser access) with precise timestamps and user attribution [15]. Audit logs are retained for extended periods to satisfy institutional compliance and legal hold requirements. Auditable records of proposal feedback exchanges (coordinator feedback, company responses timestamps) are maintained to enable dispute resolution and quality review. Superuser access is logged in detail, including the accessing administrator's identify, target user account, session timestamp, and duration, providing complete transparency into administrative actions.

3.2.4.3 Reliability & Availability

Reliability requirements ensure the platform remains operation during critical periods (e.g., proposal submissions, seriation execution). The transactional email service must guarantee high deliverability rates to ensure time-sensitive notifications (proposal deadlines, seriation results) reach intended recipients reliably, addressing the legacy system's chronic email delivery failure. The system implements automated backups of all critical data (database, uploaded documents, configuration) with off-site replication to enable recovery in case of data loss or system failure. Health monitoring and automated alerting enable rapid detection and response to critical failure (database unavailability, backup failures, email service degradation), reducing mean time to recovery.

3.2.4.4 Performance & Scalability

Performance requirements ensure the platform provides a responsive, interactive user experience even under peak load conditions. All-user facing pages (dashboards, proposal lists, candidacy tracking) must load within 3 seconds under normal load conditions.

Critical dashboards (Coordinator Edition dashboard, Company Proposal Tracker, Student Status View) must support real-time updates using WebSocket subscriptions of server-sent events, reflecting database changes without manual page refresh - a significant improvement over the legacy system's static, manual-refresh interface. API endpoints must respond within 500 ms for 95% of requests under typical load, and within 2 seconds for 99% requests, ensuring seamless client-side responsiveness. The system implements API rate limiting to protect against denial-of-service attacks and ensure fair resource allocation, limit each user to reasonable thresholds (100 requests/minute for standard operations, 300 requests/minute for bulk operations).

3.2.4.5 Usability & Accessibility

Usability requirements ensure the platform is accessible to all users, including those with disabilities, and provide an intuitive, modern user experience that addresses the legacy system's "Outdated User Experience" limitation. The user interface must be responsive and mobile-friendly, providing optimal view experience both on desktop and mobile devices. All interactive elements must provide clear visual feedback (hover states, focus indicators, loading states) to enable users to understand the current application state and response appropriately. New user workflows (registration, proposal submission, candidacy ranking) must be completable without training or external documentation, with clear inline guidance and error messages, reflecting modern web application standards.

3.2.4.6 Interoperability & Data Exchange

Interoperability requirements enable seamless data integration with institutional systems and export of data for analysis and use in external tools, addressing the legacy system's fragmentation. The system facilitates bulk import of users (students via CSV or manual entry, professors, secretariat staff) with validation, error reporting and rollback capabilities, streamlining user provisioning. The system supports export of reporting data (student lists, proposals, seriation results, internship assignments) in standard formats (CSV, JSON) for use in external tools. A documented REST API enables third-party systems to query limited, read-only data, supporting institutional integration without compromising data security.

3.2.4.7 Maintainability & Extensibility

Maintainability requirements ensure the platform can be sustained, debugged, and evolved over time without accumulating technical debt. The system architecture must be modular and layered (presentation layer, business logic, data access layer), enabling independent updates to frontend, backend, or database without cascading failures. The codebase must be documented with README files, API documentation, and inline comments for complex business logic, enabling future developers to understand and modify

the system effectively. The system supports semantic versioning and uses Git-based version control with meaningful commit message and branch-based workflows for traceability and change management. Configuration (database URLs, email credentials, feature flags) must be externalized using environment variables or configuration files, not hardcoded, to enable deployment across development, staging, and production environments.

3.2.4.8 Email Deliverability (Critical Legacy Fix)

Email deliverability is elevated to a dedicated non-functional requirement category because the legacy system's email failures represent one of the most critical operational problems identified in Section 3.1. All outbound emails must implement sender authentication protocols to minimize spam flagging and maximize delivery to user inboxes. The system must integrate with a professional transactional email service with built-in deliverability monitoring and bounce handling, enabling real-time visibility into email delivery status and rapid troubleshooting of delivery issues. Email templates must be tested and validated before deployment to ensure correct rendering across major email clients (Gmail, Outlook, Apple Mail) and responsive display on mobile devices.

3.3 Use Cases

Use case modeling is a requirements engineering technique that describes interactions between actors (stakeholders) and the system to achieve specific goals. This section documents key use cases for the dissertation/internship management platform, representing critical workflows identified in the requirements analysis. Each use case follows a standardized format including actors, preconditions, postconditions, main flow, and alternative flows.

3.3.1 Actors and Their Goals

Bases on the stakeholder analysis in Section 3.2.2, the platform involves the following actors:

Primary Actors (initiate use cases to achieve their own goals):

- Student - Seeks to discover proposals, rank preferences, and track candidacy status
- Company Representative - Aims to submit proposals and manage company participation
- Professor - Intends to propose dissertation themes and supervise students
- Edition Coordinator - Responsible for configuring editions and overseeing workflows

Secondary Actors (support the system but do not initiate the primary goal):

- Secretariat - Manage financial and legal documentation

- System Administrator - Maintains infrastructure and system health

3.3.2 High-Level Use Case Diagram

3.3.3 Detailed Use Case Specifications

3.3.3.1 UC-01: Student Ranks Internship Preferences

3.3.3.2 UC-02: Coordinator Validates and Provides Feedback on Proposal

3.3.3.3 UC-03: System Executes Student Seriation Algorithm

3.3.3.4 UC-04: Company Responds to Coordinator Feedback and Resubmits Proposal

3.3.4 Use Case Relationships and Extensions

3.3.5 Summary of Use Cases

4

Solution Proposal

4.1 Proposed System Architecture

TODO: high-level diagram of the system (interaction frontend, backend, db etc)

4.2 Process Modelling (BPMN)

TODO: mapping of workflows with bpmn

4.3 Data Model (ERD)

TODO: presentations of the ER diagram (db diagram) e definition of the entities there

4.4 API and Component Design

TODO: definition of the main endpoints and the layer logic (controller, service, repository)

4.5 Work Plan

TODO: maybe a cronogram (gantt?) with the development phases, tests, docs writing, risks and mitigations

Bibliography

[1] AlSalah. *Benefits and Drawbacks of Using Backend as a Service*. Point of SaaS. 2025. URL: https://pointofsaas.com/benefits-and-drawbacks-of-using-backend-as-a-service-3/ (cit. on p. 6).

[2] *API Routes*. Next.js Documentation. Vercel. 2025. URL: https://nextjs.org/docs/pages/building-your-application/routing/api-routes (cit. on p. 11).

[3] *App Router (Next.js)*. Next.js Documentation. Vercel. 2026. URL: https://nextjs.org/docs/app (cit. on p. 11).

[4] Ayush Sharma. *Difference between SQL and NoSQL*. GeeksforGeeks. 2025. URL: https://www.geeksforgeeks.org/sql/difference-between-sql-and-nosql/ (cit. on p. 7).

[5] B. Biskupski and A. Pytko-Włodarczyk. *Backend as a Service (BaaS) vs. custom backend: Which one to choose and why?* Accessed: 2026-01-17. 2019. URL: https://www.merixstudio.com/blog/backend-service-baas-vs-custom-backend (cit. on p. 6).

[6] *Chapter 15. Envers*. Hibernate Community. 2013. URL: https://docs.hibernate.org/orm/4.3/devguide/en-US/html/ch15.html (cit. on p. 7).

[7] P. P. Chen. "The Entity-Relationship Model: Toward a Unified View of Data". In: *ACM Transactions on Database Systems* 1.1 (1976), pp. 9–36 (cit. on p. 4).

[8] Cloudflare, Inc. *What is vibe coding?* 2025. URL: https://www.perplexity.ai/search/add-this-as-a-citation-in-late-dWWatfeETdCTpmIFNr6Jbg?sm=d (cit. on p. 13).

[9] W. contributors. *Vibe coding - chat based AI-assisted software development definition*. Accessed: 2026-01-17. 2026. URL: https://en.wikipedia.org/wiki/Vibe_coding (cit. on p. 4).

[10] David Clayton. *How to Improve Deliverability of Transactional Emails*. SMTP. 2023. URL: https://www.smtp.com/blog/transactional-email/improve-deliverability-of-transactional-emails/ (cit. on pp. 13, 15).

[11] Elena. *Why Using Legacy PHP Versions Makes Your Website Vulnerable*. Fastcomet. 2022. URL: https://www.fastcomet.com/blog/legacy-php-versions-makes-your-website-vulnerable (cit. on p. 15).

[12] *Embedded JavaScript templating Documentation*. EJS. 2026. URL: https://ejs.co/#docs (cit. on p. 7).

[13] *Express.js Documentation*. Express.js Foundation. 2026. URL: https://expressjs.com/ (cit. on p. 7).

[14] Figma. *What is Vibe Coding? Everything You Need to Know*. 2025. URL: https://www.figma.com/resource-library/what-is-vibe-coding/ (cit. on p. 5).

[15] Fortra. *Audit Log Best Practices for Security & Compliance*. Fortra. 2024. URL: https://www.fortra.com/blog/audit-log-best-practices-security-compliance (cit. on p. 22).

[16] Fred Pham. *Why Your Database Needs Boundaries: An Intro to PostgreSQL's Row Level Security (RLS)*. Shift Asia Dev blog. 2025. URL: https://shiftasia.com/community/why-your-database-needs-boundaries-an-intro-to-postgresqls-row-level-security-rls/ (cit. on p. 9).

[17] O. Glib. *Backend as a Service use Cases: How to apply*. Accessed: 2026-01-17. 2024. URL: https://acropolium.com/blog/baas-use-cases/ (cit. on p. 5).

[18] *Handlebars.js Documentation*. Handlebars.js. 2026. URL: https://handlebarsjs.com/ (cit. on p. 7).

[19] IBM Corporation. *Foreign key (referential) constraints*. IBM DB2 12.1.x Documentation. Accessed: 2026-01-21. 2026. URL: https://www.ibm.com/docs/en/db2/12.1.x?topic=constraints-foreign-key-referential (cit. on pp. 7–9).

[20] Jasmin Tankić. *Unlocking the Power of Postgres Listen/Notify: Building a Scalable Solution With Spring Boot Integration*. DZone. 2023. URL: https://dzone.com/articles/leveraging-postgres-listennotify-in-spring-boot (cit. on p. 12).

[21] kartik. *Difference between http:// and https://*. GeeksforGeeks. 2026. URL: https://www.geeksforgeeks.org/computer-networks/difference-between-http-and-https/ (cit. on pp. 15, 22).

[22] Ketevan Bostoganashvili and Piotr Malek. *I Compared 7 Best Transactional Email Services: Here's What I Found*. mailtrap. 2025. URL: https://mailtrap.io/blog/transactional-email-services/ (cit. on p. 12).

[23] Mercy Tanga. *SSR vs. SSG in Next.js: Differences, Advantages, and Use Cases*. strapi. 2024. URL: https://strapi.io/blog/ssr-vs-ssg-in-nextjs-differences-advantages-and-use-cases (cit. on p. 11).

[24] Microsoft. *Joins (SQL Server)*. SQL Server Documentation. 2025. URL: https://learn.microsoft.com/en-us/sql/relational-databases/performance/joins?view=sql-server-ver17 (cit. on p. 8).

[25] Nafisa Anjum. *ReactJS Reconciliation*. GeeksforGeeks. 2025. URL: https://www.geeksforgeeks.org/reactjs/reactjs-reconciliation/ (cit. on p. 10).

[26] *Next.js Docs*. Vercel. 2026. URL: https://nextjs.org/docs (cit. on pp. 10, 11).

[27] *Node.js Documentation*. Node.js Foundation. 2026. URL: https://nodejs.org/en/docs/ (cit. on p. 7).

[28] Object Management Group. *Business Process Model And Notation (BPMN) Version 2.0*. OMG Document Number: formal/2011-01-03. Version 2.0. Object Management Group, 2011 (cit. on p. 3).

[29] Oskar Dudycz. *PostgreSQL JSONB - Powerful Storage for Semi-Structured Data*. 2025. URL: https://www.architecture-weekly.com/p/postgresql-jsonb-powerful-storage (cit. on pp. 8, 9).

[30] Paul Bratslavsky. *10 Best WYSIWYG Editors for 2025*. strapi. 2025. URL: https://strapi.io/blog/best-wysiwyg-editors (cit. on p. 12).

[31] Pedro Alonso. *PostgreSQL LISTEN/NOTIFY: Real-Time Without the Message Broker*. 2025. URL: https://www.pedroalonso.net/blog/postgres-listen-notify-real-time/ (cit. on p. 12).

[32] Philippe Sevestre. *Using PostgreSQL as a Message Broker*. Baeldung. 2023. URL: https://www.baeldung.com/spring-postgresql-message-broker (cit. on p. 12).

[33] PostgreSQL Global Development Group. *8.14. JSON Types*. PostgreSQL 18 Documentation. 2026. URL: https://www.postgresql.org/docs/current/datatype-json.html (cit. on p. 8).

[34] Pranjal Nanda. *React Component Based Architecture*. GeeksforGeeks. 2025. URL: https://www.geeksforgeeks.org/reactjs/react-component-based-architecture/ (cit. on p. 10).

[35] K. Pykes. *What Are ACID Transactions? A Complete Guide for Beginners*. Datacamp. 2025. URL: https://www.datacamp.com/blog/acid-transactions (cit. on p. 7).

[36] React Team. *React*. Official React Documentation. Meta. 2025. URL: https://react.dev/ (cit. on p. 10).

[37] React Team. *React*. Meta Open Source Projects. Meta Open Source. 2026. URL: https://opensource.fb.com/projects/react/ (visited on 2026-01-23) (cit. on p. 10).

[38] Sachin Chaurasiya. *Tailwind CSS: Utility-First Styling for Rapid UI Development*. GeeksforGeeks. 2025. URL: https://www.geeksforgeeks.org/blogs/tailwind-css-utility-first-styling-for-rapid-ui-development/ (cit. on p. 11).

[39] J. Schibelli. *Vibe Coding: How AI is Transforming Software Development*. Accessed: 2026-01-17. 2025. URL: https://dev.to/johnschibelli/the-emergence-of-vibe-coding-how-ai-is-revolutionizing-software-development-4275 (cit. on pp. 5, 13).

[40] K. Schwaber and J. Sutherland. *The 2020 Scrum Guide*. Accessed: 2026-01-17. 2020. URL: https://scrumguides.org/ (cit. on p. 3).

[41] R. H. Shaikh. *Referential Integrity: Why It's Vital for Databases*. acceldata. 2024. URL: https://www.acceldata.io/blog/referential-integrity-why-its-vital-for-databases (cit. on pp. 8, 9).

[42] Spring Data JPA Team. *JPA*. Spring Data JPA Reference Documentation. VMware, Inc. 2026. URL: https://docs.spring.io/spring-data/jpa/reference/jpa.html (cit. on p. 9).

[43] Spring Team. *Spring Boot Reference Documentation*. 3.x. Version 3.x. VMware, Inc. 2026. URL: https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/ (cit. on pp. 4, 6, 12).

[44] Supabase. *Auth Overview*. 2026. URL: https://supabase.com/docs/guides/auth (cit. on p. 6).

[45] Supabase. *Database*. 2026. URL: https://supabase.com/docs/guides/database/overview (cit. on p. 9).

[46] Supabase. *Edge Functions*. 2026. URL: https://supabase.com/docs/guides/functions (cit. on p. 6).

[47] Supabase. *Realtime*. 2026. URL: https://supabase.com/docs/guides/realtime (cit. on p. 12).

[48] Supabase. *REST API Overview*. 2026. URL: https://supabase.com/docs/guides/api#rest-api-overview (cit. on p. 6).

[49] Supabase. *Row Level Security*. 2026. URL: https://supabase.com/docs/guides/database/postgres/row-level-security (cit. on p. 6).

[50] Supabase. *Supabase: The Open Source Firebase Alternative*. https://github.com/supabase/supabase. Project README. 2019 (cit. on pp. 4, 6).

[51] Susan Stapleton. *Role-Based Access Control (RBAC) - A Comprehensive Guide*. pathlock. 2025. URL: https://pathlock.com/blog/role-based-access-control-rbac/ (cit. on p. 22).

[52] Tomasz Dąbrowski. *Using Spring Boot for WebSocket Implementation with STOMP*. Toptal. 2026. URL: https://www.toptal.com/developers/java/stomp-spring-boot-websocket (cit. on p. 12).

[53] *What's the Difference Between an ACID and a BASE Database?* 2026. URL: https://aws.amazon.com/compare/the-difference-between-acid-and-base-database/ (cit. on p. 7).

[54] *What's the Difference Between Relational and Non-relational Databases?* 2026. URL: https://aws.amazon.com/compare/the-difference-between-relational-and-non-relational-databases/ (cit. on pp. 7, 8).

I

Annex 1 - Functional and Non-Functional Requirements

Functional Requirements

Table I.1: Functional Requirements - Edition and Configuration Management

ID	Description
FR-11	The system shall allow Edition Coordinators to create new academic editions, specifying edition type (Undergraduate Internship vs Master's Dissertation) and defining key timelines (proposal submissions deadline, interview periods, finalization deadline).
FR-12	The system shall allow Edition Coordinators to define and manage time slots for company proposal presentations, which companies can subsequently view and book during available windows
FR-13	The system shall enable Edition Coordinators to create and edit reusable email templates within the platform's interface, supporting dynamic variables (student names, company names, deadlines) and content-specific formatting for different communication stages.
FR-14	The system shall allow Edition Coordinators to create and maintain edition-specific informational pages (FAQs, Regulations, Contact Information, Important Dates) using a modern WYSIWYG editor.
FR-15	The system shall allow Edition Coordinators to publish news and announcements target at specific user roles (e.g., news visible only to Students, Companies, or Professors) within a given edition or platform-wide

Table I.2: Functional Requirements - Proposal Submission and Classification

ID	Description
FR-21	The system shall allow Companies and Professors to submit detailed proposals or dissertation themes, including attributes such as title, objectives, technical requirements, location, benefits, and supervisor information.
FR-22	The system shall implement a feedback loop allowing Edition Coordinators to request clarifications on specific aspects of proposals without full rejection, triggering email notifications to the author for editing and resubmission.
FR-23	The system shall allow Edition Coordinators to accept, conditionally accept, or reject proposals with detailed feedback comments.
FR-24	The system shall allow Companies to import or duplicate proposals from previous editions into the current edition, with the option to modify details before resubmission.
FR-25	The system shall allow Companies to indicate their intent to give oral presentations of their proposals and select available time slots directly within the platform.
FR-26	The system shall notify Companies of important deadlines (proposal submission closure, feedback response deadline, presentation scheduling deadline) via email and in-app notifications.
FR-27	The system shall notify Companies when a new edition opens in which they are eligible to submit proposals (Call for Proposals).

Table I.3: Functional Requirements - Student Candidacies and Seriation

ID	Description
FR-31	The system shall allow Students to view all available proposals for their enrolled edition and rank a specified number of preferred proposals in preference order.
FR-32	The system shall allow Students to submit supporting documentation (CV, motivation letter, recommendation letter) when registering interest in specific proposals, if required by the coordinator.
FR-33	The system shall execute an automated ranking algorithm based on predefine criteria (e.g. ECTS completed, grade average, coordinator-defined weights) to produce a seriation list of candidates for each proposal.
FR-34	The system shall notify Students of important timeline events via email and in-app notifications (registration opening, ranking deadline, seriation results, attribution confirmation)
FR-35	The system shall allow Companies to view the list of serialized (ranked) students for their proposals and manage candidate selections throughout the interview and confirmation phases.
FR-36	The system shall allow Companies to exclude or deselect candidates from their proposal's ranked list if needed.

Table I.4: Functional Requirements - Company and User Management

ID	Description
FR-41	The system shall allow Edition Coordinators to validate new company registrations, ensuring they sign the necessary legal protocols before submitting proposals
FR-42	The system shall Company Representatives to manage their organizational hierarchy, including parent company designation and sub-entities/departments, and to update company contact details and registration information
FR-43	The system shall allow Company Representatives to manage company employee accounts (add, remove, update) and designate which employees are responsible for specific proposals or serve as mentors.
FR-44	The system shall provide "Superuser" functionality, allowing Edition Coordinators to temporarily access other user accounts using only their user's identifier (student number or username) for support and troubleshooting purposes, with full audit logging of such access.
FR-45	The system shall allow Edition Coordinators to pre-register users (Students via CSV bulk import or manual entry, Professors, Secretariat staff) with role-based permissions.
FR-46	The system shall allow Edition Coordinators to assign or revoke the Sub-Coordinators role to Professors for specific reasons.

Table I.5: Functional Requirements - Communication and Notifications

ID	Description
FR-51	The system shall automate the "Call for Proposals" process, sending notifications to all registered companies (that have an active status) at the start of each edition, specifying submission deadlines and edition type.
FR-52	The system shall allow the publication of news and announcements targeted at specific user roles (e.g., news visible only to Students)
FR-53	The system shall enforce email deliverability best practices to minimize the likelihood of notifications being flagged as spam.
FR-54	The system shall support both email and in-app notifications for critical events (new proposals, feedback requests, deadline reminders, seriation results)
FR-55	The system shall allow Edition Coordinators to send targeted email communications to specific user groups (e.g., all enrolled Students, all registered Companies) with customizable subject and body content.

Table I.6: Functional Requirements - Reporting and Data Export

ID	Description
FR-61	The system shall allow Edition Coordinators and Administrators to export lists of students, proposals, candidacies, and final assignments in open standard formats (CSV, JSON).
FR-62	The system shall maintain detailed audit logs of critical system actions (user logins/logouts, proposal status changes, superuser access, permission modifications, bulk imports), with timestamps and user attribution.
FR-63	The system shall provide auditable records of all proposal feedback exchanges (coordinator feedback, company responses, timestamp of modifications) for compliance and dispute resolution.

Table I.7: Functional Requirements - Master's Dissertation Lifecycle Support

ID	Description
FR-71	The system shall support specific workflows for Master's degrees, including the distinction between scientific dissertations and engineering projects.
FR-72	The system shall allow Professors to propose scientific dissertation themes for Master's programs and manage their list of proposed themes and assigned students.
FR-73	The system shall enable tracking of dissertation progress and completing status, including milestones such as topic approval, and final submission/defense.
FR-74	The system should have edition for the Master's degree scientific dissertation and engineering projects that work the same way as the undergraduate internships, with just some changes.

Non-Functional Requirements

Table I.8: Non-Functional Requirements - Security

ID	Description
NFR-SEC-1	The system shall implement Row-Level Security at the database level to enforce strict data isolation, ensuring users can only access data they are authorized to view (e.g., companies see only their own proposals and candidates)
NFR-SEC-2	All data in transit shall be encrypted using HTTPS/TLS, preventing interception of sensitive information during transmission.
NFR-SEC-3	Password and sensitive credentials shall be hashed using industry-standard algorithms (bcrypt, PBKDF2, or Argon2) and shall never be stored in plaintext.
NFR-SEC-4	The system shall enforce role-based access control (RBAC) with granular permission assignment, ensuring users can only perform actions consistent with their assigned role.

Table I.9: Non-Functional Requirements - Auditability & Compliance

ID	Description
NFR-AUD-1	The system shall record an immutable, tamper-evident audit log of all critical actions (user logins/logouts, data modifications, status changes, permission assignments, superuser access) with precise timestamps and user attribution.
NFR-AUD-2	Audit logs shall be retained for long time to satisfy institutional compliance and legal hold requirements.
NFR-AUD-3	The system shall provide auditable records of all proposals feedback exchanges (coordinator feedback, company responses, timestamp of submission/modification) enabling dispute resolution and quality review.
NFR-AUD-4	Superuser access shall be logged in detail, including the identity of the accessing administrator, the target user account, timestamp, and duration of the session.

Table I.10: Non-Functional Requirements - Reliability & Availability

ID	Description
NFR-REL-1	The transaction email service shall guarantee a good deliverability rate to ensure time-sensitive notifications (e.g., proposals deadlines, seriation results) reach intended recipients reliably
NFR-REL-2	The system shall implement automated backups of all critical data (database, uploaded documents, configuration) with off-site replication to enable recovery in case of data loss.
NFR-REL-3	The system shall implement health monitoring and automated alerts for critical failures (database unavailability, backup failures, email service degradation), enabling rapid response by administrative staff.

Table I.11: Non-Functional Requirements - Performance & Scalability

ID	Description
NFR-PERF-1	All user-facing pages (dashboards, proposal lists, candidacy tracking) shall load within 3 seconds under normal load conditions.
NFR-PERF-2	Critical dashboards (Coordinator Edition Dashboard, Company Proposal Tracker, Student Status View) shall support real-time updates using WebSocket subscriptions or server-sent events, reflecting database changes without manual page refresh.
NFR-PERF-3	API endpoints shall respond within 500 ms for 95% of requests under typical load, and within 2 seconds for 99% of requests.
NFR-PERF-4	The system shall implement API rate limiting to protect against denial-of-service attacks and ensure fair resource allocation, limiting each user/API client to reasonable thresholds (e.g. 100 requests/minute for standard users, 300 requests/minute for bulk operations).

Table I.12: Non-Functional Requirements - Usability & Accessibility

ID	Description
NFR-USAB-1	The user interface shall be responsive and mobile-friendly, providing an optimal viewing experience on desktop (1920x1080) and mobile devices (320x568).
NFR-USAB-2	All interactive elements shall provide clear visual feedback (hover states, focus indicators, loading states) enabling users to understand the current application state.
NFR-USAB-3	New user workflows (registration, proposal submission, candidacy ranking) shall be completable without raining or external documentation, with clear inline guidance and error messages.

Table I.13: Non-Functional Requirements - Interoperability & Data Exchange

ID	Description
NFR-INTER-1	The system shall facilitate bulk import of users (Students, Professors, Secretariat) via CSV files with validation, error reporting, and rollback capabilities.
NFR-INTER-2	The system shall support export reporting data (student lists, proposals, seriation results, internship assignments) in standard formats (CSV, JSON) for use in external tools
NFR-INTER-3	The system shall provide a documented REST API for institutional integration, enabling third-party systems to query limited, read-only data.

Table I.14: Non-Functional Requirements - Maintainability & Extensibility

ID	Description
NFR-MAINT-1	The system architecture shall be modular and layered (presentation, business logic, data access), enabling independent updates to the frontend, backend, or database without cascading failures.
NFR-MAINT-2	The codebase shall be documented with README files, API documentation (Swagger/OpenAPI), and inline comments for complex business logic, enabling new developers to understand and modify the system.
NFR-MAINT-3	The system shall support semantic versioning and use Git-based version control with meaningful commit messages and branch-based workflows for traceability.
NFR-MAINT-4	Configuration (database URLs, email credentials, feature flags) shall be externalized using environment variables or configuration files, not hardcoded, to enable deployment across development, staging and production environments.

Table I.15: Non-Functional Requirements - Email Deliverability (Critical Legacy Fix)

ID	Description
NFR-EMAIL-1	All outbound emails shall implement sender authentication protocols to minimize spam flagging
NFR-EMAIL-2	The system shall integrate with a professional transaction email service with built-in deliverability monitoring and bounce handling.
NFR-EMAIL-3	Email templates shall be tested and validate before deployment to ensure correct rendering across major email clients (Gmail, Outlook, Apple Mail) and responsive display on mobile.